

IT Expense System

Beginner From-Scratch Build Tutorial

Angular frontend, Bootstrap styling, Node.js Express backend, dummy data, and future SQL Server Express integration

Beginner promise

This version explains not only what to create, but why each folder and file exists, where to put code, how components connect to services and routes, where styles belong, and how the frontend and backend talk to each other.

Read this from top to bottom if you want to rebuild the app from zero. Use the checklist at the end when reviewing your work.

1. The Big Mental Model

Before writing code, understand the moving parts. The frontend is what users see. The backend is what Angular calls when it needs login or data. The database will come later. The current dummy store pretends to be the database so we can test routes now.

```
User clicks screen
-> Angular component handles the click
-> Component calls a service
-> Service calls ApiService or local store
-> ApiService calls Express backend
-> Express middleware checks login and role
-> Express route asks dummy-store.js for data
-> JSON returns to Angular
-> Component updates the screen
```

Piece	Beginner explanation
Component	A screen or part of a screen. Example: LoginPage, RequestListPage, MainLayout.
Template HTML	The visual markup for a component. This is where buttons, forms, tables, and Bootstrap classes go.
Component CSS	Small styles that belong only to one component.
Service	Reusable logic that components call. Example: AuthService manages login. RequestStoreService manages local request data.
Model	TypeScript definitions that explain what data should look like.
Route	A URL rule that says which component appears for a path.
Guard	A route checker. It blocks users who are not logged in or do not have the right role.
Interceptor	A helper that automatically adds the auth token to backend API calls.
Backend route	An Express endpoint such as POST /api/auth/login or GET /api/expense-requests.
Middleware	Backend code that runs before route handlers, usually for auth and permissions.
Store	Temporary backend data layer. Later replaced by SQL Server repository files.

2. Why This Project Structure

The structure is split by responsibility. This is the most important habit for beginners: do not put everything in one file. Each folder answers one question.

```
src/app
core shared app logic used by many features
features actual business pages grouped by module
shared reusable layout and page components

backend
data dummy seed records
store temporary data access functions
middleware auth and role checks
routes API endpoint groups
server.js starts Express and connects everything
```

Folder	Why we choose it
src/app/core/api	All API URL and HTTP helper code lives here so components do not hard-code backend URLs.
src/app/core/auth	Login, session, token, guards, and role checks are central app concerns.
src/app/core/models	Shared TypeScript models prevent guessing what a user or request looks like.
src/app/features	Each business area gets its own folder, so Requests does not mix with Reports or Admin.
src/app/shared/layout	The sidebar/topbar layout is reused by many pages.

Folder	Why we choose it
src/app/shared/pages	Placeholder-style feature pages can reuse one component.
backend/routes	Each backend module gets its own route file, keeping server.js small.
backend/store	All temporary data operations live in one place so SQL Server can replace them later.

3. Start From Zero

Step 1: Create the Angular app

Explanation: Angular CLI creates the frontend project, TypeScript config, build config, and src folder for you.

How to do it: Run `ng new it-expense --routing --style=css`, then open the folder.

You know it works when: You see `angular.json`, `package.json`, `tsconfig` files, and `src`.

```
ng new it-expense --routing --style=css
cd it-expense
```

Step 2: Run the empty app

Explanation: Always test the generated app before adding your own code. This proves your environment works.

How to do it: Run `npm.cmd start` and open `http://localhost:4200`.

You know it works when: The Angular starter page loads.

```
npm.cmd start
```

Step 3: Create the folder layout

Explanation: Folders are like labeled drawers. You will know where every future file belongs.

How to do it: Create `core`, `features`, `shared`, and their subfolders under `src/app`.

You know it works when: Your project tree matches the structure in section 2.

4. Install Dependencies

Step 1: Install Bootstrap

Explanation: Bootstrap gives you ready-made layout, form, button, card, badge, and table classes. This keeps CSS simpler.

How to do it: Run `npm.cmd install bootstrap`.

You know it works when: `package.json` contains `bootstrap`.

```
npm.cmd install bootstrap
```

Step 2: Install Express and CORS

Explanation: Express is your backend API server. CORS allows Angular on port 4200 to call Express on port 5000.

How to do it: Run `npm.cmd install express cors`.

You know it works when: `package.json` contains `express` and `cors`.

```
npm.cmd install express cors
```

Step 3: Add helpful scripts

Explanation: Scripts let you start frontend and backend with short commands.

How to do it: Add `start:api` and `dev:api` to `package.json`.

You know it works when: `npm.cmd run start:api` starts the backend.

```
"scripts": {
  "start": "ng serve",
  "start:api": "node backend/server.js",
  "dev:api": "node --watch backend/server.js",
  "build": "ng build"
}
```

5. Where Styles Go

A beginner mistake is putting all styles everywhere. This app uses three style levels.

Style location	Use it for
Bootstrap CSS	Default UI components and utility classes. It is imported first in angular.json.
src/theme.css	Your app-wide brand variables: colors, radius, sidebar colors, Bootstrap variable overrides.
src/styles.css	Small global helper classes used across pages, such as page-title, page-subtitle, app-table, app-code.
component.css	Only styles that belong to one component, such as sidebar layout or report print behavior.

Step 1: Import Bootstrap and app styles

Explanation: Order matters. Bootstrap loads first, then your theme overrides it, then global helper styles load last.

How to do it: Edit angular.json styles array.

You know it works when: Bootstrap classes and your theme variables both work.

```
"styles": [  
  "node_modules/bootstrap/dist/css/bootstrap.min.css",  
  "src/theme.css",  
  "src/styles.css"  
]
```

Step 2: Create src/theme.css

Explanation: This is the file you customize when changing app colors. This is safer than editing every component.

How to do it: Define --app-primary, --app-bg, --app-sidebar-bg, --app-radius, and Bootstrap variables.

You know it works when: Primary buttons, badges, links, and the sidebar follow your theme.

```
:root {  
  --app-primary: #285f8f;  
  --app-accent: #00856f;  
  --app-bg: #f4f6f8;  
  --app-sidebar-bg: #17202a;  
  --app-radius: 0.5rem;  
  --bs-primary: var(--app-primary);  
}
```

Step 3: Use Bootstrap in templates

Explanation: Most layout should come from Bootstrap classes, not custom CSS.

How to do it: Use card, card-body, row, col, btn, table, form-control, form-select, badge, and alert.

You know it works when: Your component CSS becomes shorter.

6. Create Models First

Models are the agreement between your code files. They tell you what data is expected. Put shared models in src/app/core/models/domain.model.ts.

Step 1: Define user roles

Explanation: Routes, guards, and backend data all need the same role names.

How to do it: Create a UserRole type with Requester, Endorser, Approver, FinanceViewer, and Admin.

You know it works when: TypeScript helps prevent misspelled role names.

Step 2: Define request statuses

Explanation: Workflow actions depend on status names. Keeping them typed makes the workflow easier to understand.

How to do it: Create ExpenseRequestStatus with Draft, PendingEndorsement, PendingApproval, Approved, Returned, Rejected, Cancelled, Closed.

You know it works when: Components can only use approved status values.

Step 3: Define interfaces

Explanation: Interfaces describe object shapes. They make component and service code easier to read.

How to do it: Create `ExpenseRequest`, `ExpenseLineItem`, and `WorkflowLogEntry` interfaces.

You know it works when: When you access `request.totalAmount`, TypeScript knows it is a number.

```
export interface ExpenseRequest {
  id: string;
  requestNumber: string;
  title: string;
  status: ExpenseRequestStatus;
  totalAmount: number;
}
```

7. Build Routing

Routes connect URLs to components. Think of routes as a table that says: if the browser path is `/requests`, show `RequestListPage`.

Step 1: Create top-level routes

Explanation: The top-level route file controls the main app navigation.

How to do it: In `app.routes.ts`, put `/auth` outside the main layout and protected feature routes inside `MainLayout`.

You know it works when: Unauthenticated users go to login; authenticated users see the sidebar layout.

Step 2: Create feature routes

Explanation: Each feature owns its own routing file so `app.routes.ts` stays readable.

How to do it: Put request routes in `src/app/features/requests/requests.routes.ts` and dashboard routes in `dashboard.routes.ts`.

You know it works when: The Requests feature can be understood without opening every other feature folder.

Step 3: Add role data to routes

Explanation: Routes should say who is allowed to open them.

How to do it: Use `data: { roles: [...] }` on routes.

You know it works when: `roleGuard` can read the route and decide whether to allow access.

```
{ path: 'requests/new', component: RequestFormPage, data: { roles: ['Requester', 'Admin'] } }
```

8. Build Auth Step By Step

Auth is split into small files because each file has a different job.

File	Job
<code>session.model.ts</code>	Defines <code>LoginCommand</code> , <code>AuthSession</code> , <code>CurrentUser</code> , and <code>localStorage</code> key.
<code>auth.service.ts</code>	Logs in, logs out, stores session, exposes current user and roles.
<code>auth.guard.ts</code>	Blocks private routes when not logged in.
<code>role.guard.ts</code>	Blocks routes when the user lacks the required role.
<code>auth-token.interceptor.ts</code>	Adds Authorization: Bearer token to backend HTTP requests.
<code>mock-auth-accounts.ts</code>	Temporary login accounts for development/testing.

Step 1: Create the session model

Explanation: Before writing login code, decide what login returns.

How to do it: Create `AuthSession` with `accessToken`, optional `refreshToken`, `expiresAt`, and `user`.

You know it works when: `AuthService` can store one predictable object after login.

Step 2: Create AuthService

Explanation: The login page should not manually touch localStorage or tokens. AuthService owns that logic.

How to do it: Inject ApiService and Router. Add login, logout, hasAnyRole, storeSession, and loadSession.

You know it works when: After login, user and roles are available everywhere.

Step 3: Create guards

Explanation: Guards protect routes before components load.

How to do it: authGuard checks isAuthenticated. roleGuard checks route data roles.

You know it works when: Users without permission are redirected.

Step 4: Create interceptor

Explanation: You do not want to manually add the token in every API call.

How to do it: Read session from localStorage and clone requests with Authorization header.

You know it works when: Protected backend routes work after login.

9. Build Components And How They Connect

A component usually has three files: TypeScript for logic, HTML for UI, and CSS for local styles. Services hold shared logic. Routes decide when components appear.

Component	How it connects
LoginPage	Template has form. TypeScript calls AuthService.login. AuthService calls backend /auth/login.
MainLayout	Template shows sidebar/topbar. It uses AuthService for current user and logout.
FeaturePage	Reads route data and MockDataService to show placeholder module content.
RequestListPage	Uses AuthService for roles and RequestStoreService for visible requests/actions.
RequestFormPage	Uses ReactiveFormsModule and RequestStoreService.create to save draft or submit.
ApprovalReportPage	Uses RequestStoreService data and computed signals for filters, totals, CSV, and print.

Step 1: Create LoginPage

Explanation: This is the first screen users see.

How to do it: Put the form in login-page.html, form logic in login-page.ts, and small hover/layout styles in login-page.css.

You know it works when: Submitting valid credentials moves to dashboard.

Step 2: Create MainLayout

Explanation: This prevents repeating sidebar and header on every page.

How to do it: Put navigation links in main-layout.html and sidebar layout styles in main-layout.css.

You know it works when: All protected pages render inside router-outlet.

Step 3: Create reusable FeaturePage

Explanation: Many modules are placeholders for now. One reusable component avoids duplicate code.

How to do it: Use route data for title, subtitle, capabilities, mockKey, and endpoints.

You know it works when: Changing a route's data changes the displayed feature page.

10. Build The Request Feature

Requests are the first real workflow. For now, Angular stores them in localStorage so you can practice the UI flow before the backend database is ready.

Step 1: Create RequestStoreService

Explanation: This service acts like a mini frontend database.

How to do it: Put it in `src/app/features/requests/request-store.service.ts`. Store requests and audit logs in `localStorage`.

You know it works when: Requests remain after browser refresh.

Step 2: Create workflow methods

Explanation: Workflow actions are easier to test when each action is a method.

How to do it: Add `create`, `submit`, `cancel`, `endorse`, `returnRequest`, `reject`, `approve`, and `close`.

You know it works when: Clicking action buttons changes request status.

Step 3: Add role filtering

Explanation: Users should only see requests relevant to their role.

How to do it: Create `visibleRequests` computed signal based on current user roles.

You know it works when: Requester, Endorser, Approver, Finance, and Admin see different lists.

Step 4: Build RequestListPage

Explanation: The list is the user's main request view.

How to do it: Use Bootstrap table, badges, and action buttons. Call store methods from button clicks.

You know it works when: The table shows status and available actions.

Step 5: Build RequestFormPage

Explanation: This is how a Requester creates a request.

How to do it: Use `ReactiveFormsModule`, Bootstrap `form-control/form-select`, and validators.

You know it works when: Save Draft and Submit Request both create records.

Frontend file	What to put there
<code>request-list-page.ts</code>	Inject services, expose requests, decide actions, run actions.
<code>request-list-page.html</code>	Bootstrap table, buttons, helper cards.
<code>request-form-page.ts</code>	Reactive form setup, total getter, submit/save methods.
<code>request-form-page.html</code>	Bootstrap form fields and action buttons.
<code>request-store.service.ts</code>	Data, <code>localStorage</code> , workflow transitions, role visibility, audit logs.

11. Build The Report Screen

The approval report shows how to build a more advanced Angular page with filters, computed values, exports, and print output.

Step 1: Create filter signals

Explanation: Signals hold current UI choices and automatically update computed values.

How to do it: Create `departmentFilter`, `statusFilter`, `groupByRequester`, and `selectedColumnKeys`.

You know it works when: Changing a select or checkbox updates the report.

Step 2: Create computed report data

Explanation: Computed values keep report logic out of the template.

How to do it: Create `filteredRequests`, `groupedRequests`, `totalAmount`, `approvedAmount`, and `pendingAmount`.

You know it works when: Summary cards and table rows match the filters.

Step 3: Create export actions

Explanation: Reports often need export/print actions.

How to do it: Build exportCsv and printPdf methods.

You know it works when: CSV downloads and print window opens.

Step 4: Style for screen and print

Explanation: Screen UI and print output have different needs.

How to do it: Use Bootstrap for screen controls and @media print in component CSS for print cleanup.

You know it works when: Toolbar hides during printing.

12. Backend Structure Explained

The backend is intentionally separate from Angular. It has its own folder because it runs as a different process.

Backend file/folder	What belongs there
backend/server.js	Creates Express app, adds CORS/JSON middleware, mounts route files, starts port 5000.
backend/data/dummy-data.js	Seed users, departments, categories, requests, and audit logs.
backend/store/dummy-store.js	Functions that read/change dummy data. Later this becomes SQL repository logic.
backend/middleware/auth.middleware.js	requireAuth and requireAnyRole checks.
backend/routes/auth.routes.js	Login, logout, refresh, me.
backend/routes/requests.routes.js	Request list, detail, draft, submit, cancel, line items.
backend/routes/workflow.routes.js	Endorse, approve, return, reject, audit trail.
backend/routes/reference.routes.js	Users, departments, cost centers, categories, projects.
backend/routes/reports.routes.js	Dashboard totals, expense summary, CSV export.
backend/routes/misc.routes.js	Health check and future placeholder endpoints.

Step 1: Create server.js

Explanation: server.js is the backend entry point. Keep it simple.

How to do it: Require route files and mount them with app.use.

You know it works when: Console prints IT Expense API running at http://localhost:5000/api.

```
app.use('/api/auth', authRoutes);
app.use('/api/expense-requests', requestRoutes);
app.use('/api/workflow', workflowRoutes);
app.use('/api/admin', referenceRoutes);
app.use('/api/reports', reportRoutes);
app.use('/api', miscRoutes);
```

Step 2: Create dummy data

Explanation: You need data before SQL Server exists.

How to do it: Create users, departments, costCenters, categories, projects, requests, and auditLogs arrays.

You know it works when: Login and request list routes can return real-looking data.

Step 3: Create dummy store

Explanation: Routes should not directly manipulate arrays. The store is the data access layer.

How to do it: Add functions such as findUserByEmail, listRequestsForUser, createRequest, transitionRequest, and auditTrail.

You know it works when: Routes stay short and readable.

13. Middleware And Routes Explained

Middleware is a checkpoint. A route is the final destination. For example, a request to approve an expense must pass login check and role check before it reaches the approve handler.

```
POST /api/workflow/requests/:id/approve
-> requireAuth checks token
-> requireAnyRole(['Approver', 'Admin']) checks permission
-> route handler calls transitionRequest
-> backend returns updated request
```

Step 1: Build requireAuth

Explanation: Protected data should not be available without login.

How to do it: Read Authorization header, decode dummy token, find user, attach req.user.

You know it works when: No token returns 401.

Step 2: Build requireAnyRole

Explanation: Not every logged-in user can do every action.

How to do it: Check if req.user.roles contains at least one allowed role.

You know it works when: Wrong role returns 403.

Step 3: Build auth routes

Explanation: Angular login needs a backend endpoint.

How to do it: POST /api/auth/login checks email/password and returns token plus user.

You know it works when: LoginPage can sign in through AuthService.

Step 4: Build request and workflow routes

Explanation: These routes are the future backend version of the request workflow.

How to do it: Add list, create draft, submit, cancel, endorse, approve, return, reject, and audit trail endpoints.

You know it works when: You can test them with PowerShell or Postman.

14. How Frontend And Backend Connect

Angular does not import backend files. It talks to the backend over HTTP using URLs. That is why ApiEndpoints and ApiService matter.

Frontend concept	Backend partner
environment.apiUrl	Base backend URL: http://localhost:5000/api
ApiEndpoints.auth.login	POST /api/auth/login
AuthService.login	Calls ApiService.post with login command
authTokenInterceptor	Adds Authorization header to backend requests
ApiEndpoints.requests.list	GET /api/expense-requests
ApiEndpoints.workflow.approve	POST /api/workflow/requests/:id/approve

Step 1: Set environment.development.ts

Explanation: This tells Angular where the backend is.

How to do it: Set apiUrl to http://localhost:5000/api and mockAuth to false.

You know it works when: Login requires the Express backend.

Step 2: Start both servers

Explanation: Frontend and backend are separate.

How to do it: Use two terminals: npm.cmd run start:api and npm.cmd start.

You know it works when: Backend health and Angular login both work.

```
# Terminal 1
npm.cmd run start:api

# Terminal 2
npm.cmd start
```

Step 3: Understand current limitation

Explanation: Auth uses the backend now, but request screens still use localStorage for workflow testing.

How to do it: Later replace RequestStoreService with an HTTP-backed request service.

You know it works when: After SQL Server is added, data will persist on the backend.

15. Testing For Beginners

Test in layers. Do not wait until the end to test everything.

Test	What it proves
npm.cmd run build	Angular templates, TypeScript, and CSS imports compile.
http://localhost:5000/api/health	Express backend is running.
POST /api/auth/login	Dummy users and auth route work.
GET /api/expense-requests with token	Middleware and protected route work.
Login in browser	Angular AuthService can talk to backend.
Different dummy accounts	Role navigation and request visibility behave correctly.
Resize browser	Bootstrap layout is responsive.

```
$login = Invoke-RestMethod -Method Post `
-Uri http://localhost:5000/api/auth/login `
-ContentType 'application/json' `
-Body '{"email":"super@test.com","password":"password123"}'

$headers = @{ Authorization = "Bearer $($login.accessToken)" }
Invoke-RestMethod -Uri http://localhost:5000/api/expense-requests -Headers $headers
```

16. Future SQL Server Express Plan

When you are ready for SQL Server Express, do not put SQL directly inside route files. Keep the same routes, but replace dummy-store.js with repository files.

```
backend
db
sql-server.js
repositories
user.repository.js
request.repository.js
workflow.repository.js
reference.repository.js
services
auth.service.js
```

```
request.service.js  
workflow.service.js
```

SQL table	Why it exists
Users	Stores login identity and profile.
Roles	Stores role names.
UserRoles	Connects users to one or more roles.
Departments	Department setup.
CostCenters	Finance cost center setup.
ExpenseCategories	Hardware, software, cloud, and GL mapping.
ExpenseRequests	Request header data and status.
ExpenseLineItems	Items inside a request.
WorkflowAuditLogs	Append-only history of actions.
ApprovalMatrix	Rules for endorsement and approval routing.
Attachments	File metadata and storage references.

Important production rule: the backend should own request numbering, allowed transitions, permissions, audit logs, and budget checks. The frontend can show buttons, but the backend must still verify everything.

17. Final Beginner Build Order

Order	Build step
1	Install Node.js and Angular CLI.
2	Create Angular project.
3	Create folder structure.
4	Add Bootstrap, theme.css, and global styles.
5	Create shared models.
6	Create routes and guards.
7	Create AuthService and login page.
8	Create MainLayout and navigation.
9	Create ApiService and ApiEndpoints.
10	Create MockDataService and FeaturePage.
11	Create RequestStoreService.
12	Create request list and request form.
13	Create approval report.
14	Create backend/server.js.
15	Create dummy-data.js and dummy-store.js.
16	Create middleware.
17	Create backend route files.
18	Set mockAuth false and test backend login.
19	Run build and browser checks.

Order	Build step
20	Later, replace dummy store with SQL Server repositories.

18. Recommended Learning Resources

Topic	Resource
JavaScript basics	MDN Dynamic scripting with JavaScript - https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting
Node.js basics	Node.js Learn introduction - https://nodejs.org/learn/getting-started/introduction-to-nodejs
Angular basics	Angular official tutorials - https://angular.dev/tutorials
Express install	Express installing guide - https://expressjs.com/en/starter/installing/
Express routing	Express routing guide - https://expressjs.com/en/guide/routing/
Express middleware	Express middleware guide - https://expressjs.com/en/guide/using-middleware.html
Bootstrap basics	Bootstrap getting started - https://getbootstrap.com/docs/5.3/getting-started/introduction/
Bootstrap customization	Bootstrap CSS variables - https://getbootstrap.com/docs/5.3/customize/css-variables/
SQL Server later	Microsoft SQL Server tutorials - https://learn.microsoft.com/en-us/sql/sql-server/tutorials-for-sql-server-2016

19. Quick Commands

```
# Start backend
npm.cmd run start:api

# Start frontend
npm.cmd start

# Build Angular
npm.cmd run build

# Test backend in browser
http://localhost:5000/api/health

# Open Angular app
http://localhost:4200
```

End of beginner guide.